

CS 411
Spring 2025
Databased

Final Report + Video

Nehan Tarefdar, Roy Zhao, Atsi Gupta, and Somrishi Banerjee
nehant2, royzhao2, atsig2, and sb91
TA: Aadarsh Hegde

Introduction

IlliniSpaces is a web-based platform designed to help students and organizations at the University of Illinois efficiently locate and informally reserve available meeting rooms on campus. By integrating university scheduling data and enabling customizable filters, the application provides a streamlined alternative to existing tools. This report outlines the development process, technical decisions, and final outcomes of the project, highlighting key changes from the original proposal and areas for future improvement.

Final Video link: https://youtu.be/MKAqZ2_Jfng

Please list out changes in the directions of your project if the final project is different from your original proposal (based on your stage 1 proposal submission).

- We made small but important schema changes
 - converting building and room identifiers from INT to VARCHAR to better reflect real-world data (some room numbers have alphabetical letters)
- The UI design was completely overhauled from a mobile design to a desktop interface.
- We replaced our original authentication approach with Clerk to simplify login management and ensure email domain enforcement.
- We did not incorporate RSO event data, which was initially considered for future integration.
- Calendar-based room availability was replaced by a simpler reservation list format.
- We expanded our scraping toolset and data collection logic to improve the quality and completeness of our dataset.
- The original division of frontend/backend responsibilities shifted over time, with all team members contributing across multiple areas based on evolving project needs.

Discuss what you think your application achieved or failed to achieve regarding its usefulness.

IlliniSpaces achieved its primary goal of offering students a centralized and user-friendly way to find available meeting rooms on campus. The search and filtering functionalities made it easy for users to find spaces based on location, capacity, and time, offering a more accessible and intuitive experience compared to platforms like SODA. While we were able to incorporate university event and course data, the updates were not real-time due to batch scraping. Additionally, the exclusion of RSO event data slightly limited the scope of availability tracking. Still, the application made meaningful progress toward solving the common problem of informal and conflicting room usage.

Discuss if you changed the schema or source of the data for your application

We made small but meaningful changes to our schema, such as converting BuildingId and RoomNumber from INT to VARCHAR to better match real-world data formats. Our data sources remained consistent with the proposal, relying on resources like courses.illinois.edu, one.illinois.edu, and public scheduling documents, although we added extra scraping logic and validation to handle missing or incomplete entries more gracefully. We also later added registrar.illinois.edu as a source to find unique yet intuitive IDs for all buildings on campus. We also added a “type” field to the Rooms table for additional information about the room being reserved.

Discuss what you change to your ER diagram and/or your table implementations. What are some differences between the original design and the final design? Why? What do you think is a more suitable design?

Although our ER diagram and table implementations largely followed our original design, specific refinements improved our database structure. We explicitly implemented cascade delete and update constraints across related tables, such as Users, Rooms, and Buildings, to efficiently manage data integrity. Additionally, we set up transactions to enforce reservation constraints, ensuring no overlapping reservations occurred across UserReservations and HardReservations. Originally, we planned for these checks at the application level, but implementing them directly within the database proved more reliable. The final design incorporating these database-level constraints and alphanumeric identifiers better reflects real-world scenarios and significantly enhances maintainability, making it a more suitable choice for our application.

Discuss what functionalities you added or removed. Why?

Over the course of the project, we made several adjustments to the functionalities we originally planned. One major change was replacing the calendar-based availability view with a simpler list format. While a calendar interface was initially intended to help users visualize room schedules more intuitively, we found that implementing and populating it accurately with data from multiple sources was more complex and time-consuming than anticipated. The list view provided a more straightforward alternative that still met user needs. We also removed the feature for generating highlighted walking directions between buildings. Although this was part of our creative component, integrating custom routing logic using external mapping tools proved to be a significant technical challenge, and we decided to prioritize other core features. In order to fulfill the creative component, however, we still implemented an interactive map that filters rooms based on distance, and we added the ability for users to create and delete favorite buildings. On the other hand, we added new functionality around data parsing and validation, such as logic to generate unique IDs for buildings missing standardized codes and the ability to filter out incomplete data entries. These additions helped ensure our database remained consistent and reliable, even when working with imperfect real-world data.

Explain how you think your advanced database programs complement your application.

Our advanced database programs significantly enhanced the functionality and usability of our IlliniSpaces application. By leveraging sophisticated SQL queries involving operations like UNION, JOIN, aggregation (GROUP BY), and nested subqueries, we effectively supported key features such as real-time availability searches, space reservations, and interactive visualizations. For instance, our UNION operations allowed precise identification of available rooms by joining data from scheduled classes and existing user reservations. Aggregations enabled grouping available rooms by building, directly enhancing the user experience through a visually intuitive map interface. An added nested subquery in a stored procedure for fetching Rooms allowed users to see the occupancy status of rooms directly on the dashboard. Overall, these advanced database techniques complemented our application by ensuring efficient data retrieval, accurate real-time updates, and a streamlined user interface, making IlliniSpaces both robust and user-friendly.

Each team member should describe one technical challenge that the team encountered. This should be sufficiently detailed such that another future team could use this as helpful advice if they were to start a similar project or where to maintain your project.

Atsi: One technical challenge we encountered involved populating our database endpoints with parsed data from university sources. Specifically, some buildings in our datasets lacked standardized building codes, which our schema required as a primary key (BuildingId). To handle this, I implemented a solution in our parsing script that generated a unique, random alphanumeric identifier for each building missing a code, ensuring these buildings could still be included in the database without violating primary key constraints. Additionally, certain data entries were incomplete or contained missing attributes, such as events missing start times, which could compromise database integrity. To address this, I developed conditional logic within our parsing endpoints to detect and skip these incomplete or invalid entries. In the end we fixed the issue with parsing our data as to not need the randomized letters for the (BuildingId), but this was definitely an interesting technical challenge to fix.

Nehan: One particular challenge that we needed to deal with was our authentication system's connection to our backend. Originally, as part of a design decision we decided to use Clerk to maintain and authenticate users using methods such as Google OAuth. However, since user information and identification was necessary on the backend, we also needed to store and be able to connect the Clerk JWT (JSON Web Token) to our user on the backend. Luckily, in our schema we defined Emails to be unique, and thus they could be used to connect a user to their Clerk token. For any API endpoint that made sensitive reads or writes to the database, we implemented logic in the function get_email on the backend which allowed us to trade a Clerk JWT for a "verified token" object using a lower-level function included in the Clerk Backend API Python package. If the verify token function fails, then the request is not authenticated and is stopped. For a successful JWT decode, this "verified token" object contains various important

fields including a user identifier for Clerk, which can be sent to another Clerk API to obtain information about the user such as their email, which we can search for in the database to get a user's unique UID. The API calls to Clerk on the backend were verified by Clerk using a secret key connected to our Clerk account, which is stored as an environment variable both in a .env file locally as well as in the Google Cloud Run environment variables for production.

Roy: One challenge is getting a comprehensive list of usable rooms on campus. We tried to find this data online but had no success. Furthermore, a raw list of every room in a building would not be very useful—we should only include rooms dedicated to learning purposes, not service cabins, restrooms, or offices. Eventually, we found a creative solution for this problem: since each course section on the course catalog explicitly shows its meeting location—building name and room number—on the course catalog, it is possible to collect a list of all operating classrooms by going through the class list. Hence, we made a trigger on HardReservations, the relation holding the classes data. As we insert class data to HardReservations, whenever we encounter an unseen room used by a class, we automatically insert the room number and the ID of its building into the Rooms relation. In this way, we gathered 805 rooms in 105 buildings, where LCLB (Lit, Cultures, & Ling) has a max of 38 rooms for classes, followed by the ECEB and Gregory Hall with 29 rooms each for classes.

Somrishi: One technical challenge we faced was populating the UserReservations table with over 1000 entries to show a proof of concept. When actually used, this table will contain the start and end times of a reservation, along with the date, building id, and room number, and works fine with users. However, to show this works, we needed to fake some user data. We created a script to fake some time in H:M format so that it could be compared against other tables. In addition, we made the start and end dates within a week of current time to ensure there were no issues with certain rooms not being available in the past or future. To ensure the foreign key constraints with the buildings and rooms, we scraped the rooms table with a SQL command and selected random values. Even though we needed multiple UserReservations, the script is only once and the command is only run once, upon which the values selected are stored in the cursor, so we do not need to do multiple selects each time to populate the table with data, which would cause the program to be slow.

Are there other things that changed comparing the final application with the original proposal?

Several aspects of the final application changed compared to our original proposal. We shifted our frontend design from a mobile-first approach to one optimized for desktop and laptop use, which better supported features like the interactive map and advanced filtering options. Instead of implementing a full calendar-based scheduling system, we opted for a simplified list view to display upcoming reservations, which allowed for faster development and clearer presentation. Highlighted walking directions were also removed from the scope due to time constraints and the added complexity of custom routing integration. For authentication, we used Clerk to enforce illinois.edu email verification, replacing our initial plan to use Google OAuth,

as Clerk provided a more streamlined and manageable solution. We also chose not to include RSO data, which was originally considered, after determining it was too complex and difficult to access. Lastly, the project ended up placing greater emphasis on data scraping and backend development than we originally anticipated, largely due to the challenges of sourcing and cleaning inconsistent external data.

Describe future work that you think, other than the interface, that the application can improve on

Beyond improving the interface, there are several key areas where the application could be enhanced in future iterations. One major opportunity is integrating real-time data synchronization instead of relying on periodic scraping and batch updates. Implementing APIs or data pipelines that automatically reflect schedule changes as they happen would make the availability information significantly more accurate and timely.

Additionally, expanding support for recurring reservations and better handling of complex scheduling patterns (like alternating weeks or irregular class meetings) would align the system more closely with real university scheduling behavior. Lastly, incorporating RSO booking data and permissions could turn the platform into a more comprehensive scheduling hub, supporting both individual students and official student organizations with varying access levels and reservation rights. Due to time constraints, we also weren't able to implement certain planned features such as room calendars and building modals on the map tab. These improvements would help transition IlliniSpaces from a helpful utility to a fully integrated part of the campus scheduling infrastructure.

Describe the final division of labor and how well you managed teamwork.

We met weekly as a group to ensure everyone remained aligned and to plan tasks for the upcoming week. Initially, we divided responsibilities into two frontend and two backend roles. However, this quickly evolved as we realized the need for all members to actively engage with the database aspects of the project. A clear example of our collaborative approach was during Checkpoint 3, where each team member individually contributed one advanced query, reflecting effective teamwork and shared involvement in database development.